

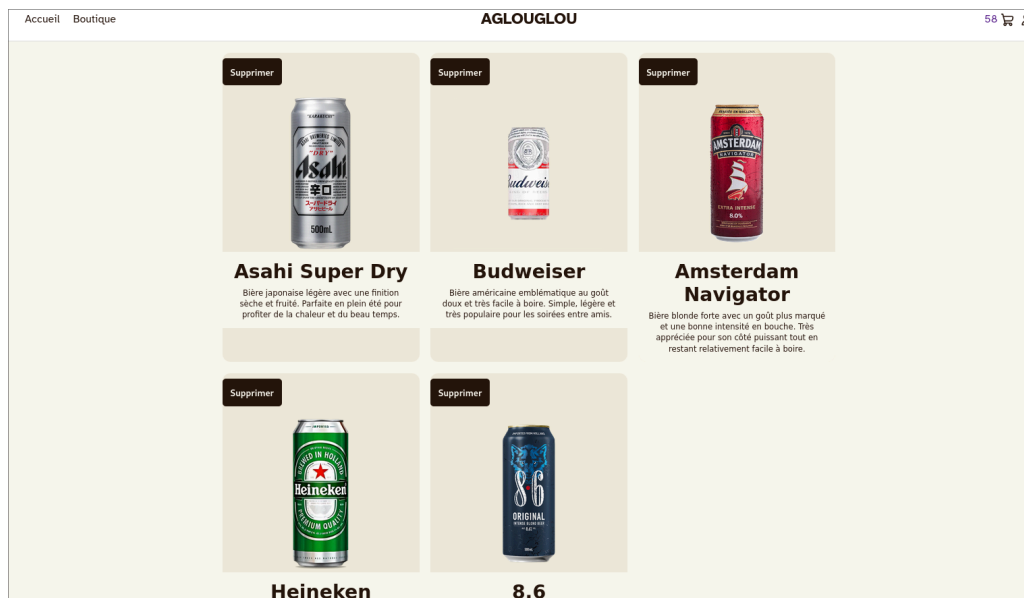
# Compte rendu du projet WEB CMS

Tom GONGORA et Théo KREDER-BAIL

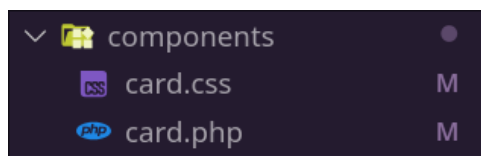
## La logique en PHP

### Utilité d'un composant

Pour faire le site il y a quelques éléments de logique en PHP intéressants à expliquer. Pour commencer, certains éléments sont gérés comme des composants : si on doit utiliser un élément plusieurs fois alors on fait un fichier pour cet élément. Par exemple dans la page de boutique chaque carte contenant un produit est appelé par une fonction :



Cette fonction est stockée dans le fichier PHP dans un dossier component



On utilise une fonction pour pouvoir utiliser des variables facilement attribuable dans le code

```

1 function Card($title, $description, $image, $slug) {
2     ?>
3
4
5     <div class="card">
6         <form method="POST" class="delete-form">
7             <input type="hidden" name="slug" value="<?> $slug ?>"> 
8             <button type="submit" class="delete-btn" onclick="alert('Vous aller supprimer un élément de la BDD')">Supprimer</button>
9         </form>
10
11         <a href="/product-page.php?slug=<?> $slug ?>">
12
13             <div class="card-image">
14                  $title ?>">
15             </div>
16
17             <div class="card-content">
18                 <h2><?> $title ?></h2>
19                 <p><?> $description ?></p>
20             </div>
21         </a>
22     </div>
23
24
25     <?php } ?>

```

Pour l'appeler il suffit de faire

```

1 foreach ($products as $product) {
2     Card($product["title"], $product["description"], $product["image"], $product["slug"]);
3 }
4 ?>

```

Avec ici un foreach pour afficher toutes les cartes.

## Le système de “slug”

Pour référencer les produits de la base de donnée au site on utilise un slug (=le nom du produit écrit de manière interprétable, par exemple pour Bière Amsterdam le slug est biere-amsterdam). Le slug sert pour afficher la page d'un produit, il est mis directement dans l'url et est récupéré par la page pour afficher le produit dans la base de donnée correspondant :

```

1 // Récupère le slug de l'url
2 $slug = $_GET["slug"];
3 // Récupère le produit associé dans la db
4 $sql = "SELECT * FROM products WHERE slug='$slug'";
5 $resultat = qdb($sql);
6 $product = mysqli_fetch_assoc($resultat);
7

```

Donc pour [http://localhost/Projet\\_Web/pages/product-page.php?slug=asahi-super-dry](http://localhost/Projet_Web/pages/product-page.php?slug=asahi-super-dry) On va chercher dans la base de donnée le produit qui à un slug correspondant à asahi-super-dry.

## La logique panier

Pour que les produits enregistrés dans le panier ne disparaissent pas au changement de page, on utilise le système de SESSION. C'est un endroit où on peut stocker des variables sous forme de tableau associatif (similaire aux dictionnaires en Python) et qui reste actif tant que le navigateur est ouvert. On peut donc quitter le site puis y revenir et le panier sera toujours enregistré. Pour utiliser la session il faut l'initialiser dans un fichier qui sera appelé par tous les fichiers PHP où on souhaite avoir accès au panier (dans notre site, le panier est affiché dynamiquement dans la barre de navigation donc toutes les pages demandent ce fichier php très simple) :

```
1 require_once "../api/add_to_cart.php"; // Pour gérer le panier
```

*Ici on appelle le fichier*

```
1 <?php
2     session_start();
3
4     if (!isset($_SESSION["cart"])) {
5         $_SESSION["cart"] = [];
6     }
7     ?>
```

*Le contenu du fichier*

## La logique en JS

### Simulation d'un achat

Une fois dans le panier, l'utilisateur peut cliquer sur le bouton "Acheter maintenant", ce bouton simule un achat grâce au Javascript

```

1 let boutonAchat = document.querySelector(".buy-button");
2
3 if (boutonAchat) {
4   boutonAchat.addEventListener("click", function () {
5     // On bloque le bouton pour éviter les clics multiples comme on va utiliser le bouton pour une animation
6     boutonAchat.disabled = true;
7     boutonAchat.style.opacity = "0.6";
8     boutonAchat.style.cursor = "not-allowed";
9
10    // On change le texte du bouton pour simuler le traitement avec la banque
11    boutonAchat.innerHTML = "Connexion à la banque... ⌚";
12
13    // On utilise setTimeout pour attendre 2 secondes
14    setTimeout(function () {
15      // Calcul d'une fausse date de livraison (Date du jour + 3 jours)
16      let dateLivraison = new Date();
17      dateLivraison.setDate(dateLivraison.getDate() + 3);
18
19      // On formate la date en format français
20      let dateFormatee = dateLivraison.toLocaleDateString("fr-FR");
21
22      // On affiche le message de confirmation avec la date calculée
23      alert(
24        "👉 Paiement accepté ! \n\nVos bières sont en cours de préparation. La livraison est estimée au : " +
25        dateFormatee +
26        ".\n\nMerci pour votre commande !"
27      );
28
29      // On affiche la validation sur le bouton
30      boutonAchat.innerHTML = "Commande validée ✅";
31      boutonAchat.style.backgroundColor = "#4CAF50"; // On le met en vert pour marquer le succès
32      boutonAchat.style.color = "white";
33    }, 2000); // 2000ms = 2 secondes
34  });
35 }

```

On utilise la fonction `setTimeout` pour faire patienter l'utilisateur 2 secondes avec un message "Connexion à la banque" puis on lui affiche une `alert()` confirmant sa commande. Il ne peut pas se tromper et passer commande 2 fois de suite, on modifie le bouton d'achat pour lui en informer

## Validation des données

Quand on ajoute une quantité d'un produit au panier on utilise le JS pour savoir si cette quantité est valide, elle doit être supérieure à 0 (sinon on remet la quantité à 1) et si elle dépasse 50 on affiche un message :

```

1 let champQuantite = document.getElementById("qty");
2
3 if (champQuantite) {
4   champQuantite.addEventListener("input", function () {
5     // On convertit la valeur du input en int
6     let valeurSaisie = parseInt(champQuantite.value);
7     // si l'utilisateur tape un nombre négatif ou 0 on remet à 1
8     if (valeurSaisie < 1) {
9       champQuantite.value = 1;
10    }
11
12    // Si plus de 50 bières on affiche un message
13    if (valeurSaisie > 49) {
14      alert("50 bières ? Vous devez être bien déshydraté !");
15    }
16  });
17 }
18

```